



AI Career Roadmap Generator: AI-Powered Career Guidance & Planning Platform

Project Description:

AI Career Roadmap Generator harnesses Generative AI to provide intelligent career guidance, offering users a fast and personalized career planning experience. The platform includes a Resume Analyzer module, which computes ATS compliance scores and extracts technical and soft skills, a Skill Gap Evaluator, which identifies missing tools and certifications for target roles, a Dynamic Career Roadmap Generator, which delivers month-by-month actionable learning plans, and a Portfolio & Certification Advisor, which suggests high-ROI certifications and project builds.

Utilizing Groq's API with the LLaMA 3.3-70B Versatile model, AI Career Roadmap Generator processes user inputs to deliver fast, high-quality career guidance content. Built with Flask and connected to MongoDB Atlas, the platform ensures a seamless and user-friendly experience. With secure API key management via .env and responsible input handling, AI Career Roadmap Generator empowers job seekers, students, and professionals to navigate career growth with confidence.

Scenarios

Scenario 1: A user uploads their PDF resume and selects Full Stack Developer as their target role. The system parses their resume, extracts current skills, identifies missing backend skills, and generates a detailed month-by-month career roadmap in seconds.

Scenario 2: A software engineer seeking transition into Machine Learning selects Data Scientist as their target role. AI Career Roadmap Generator identifies skill gaps, predicts readiness scores, and recommends specific courses and certifications.

Scenario 3: A candidate preparing for technical interviews uses the AI mentor. The system generates role-specific interview questions, behavioral tips, and system design advice to maximize preparation efficiency.

Technical Architecture:

Description: AI Career Roadmap Generator utilizes a modular three-tier architecture to deliver rapid, AI-driven content generation and career pathing. The frontend provides a responsive user interface built with HTML, CSS, and JavaScript while the Flask-based backend orchestrates data validation, user authentication, and prompt construction. It interfaces with the Groq API using the LLaMA 3.3-70B model to generate tailored career content, backed by a persistent MongoDB Atlas cloud database instance for user profile and roadmap storage.

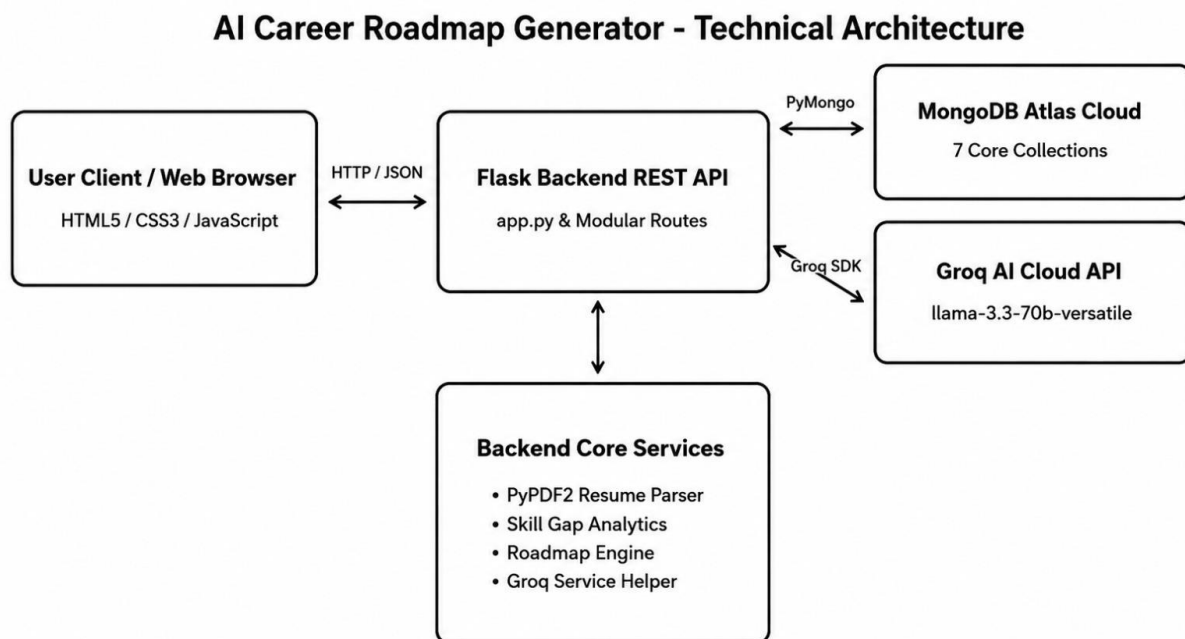


Figure: Technical Architecture Diagram of AI Career Roadmap Generator

Pre-requisites:

- **Flask Framework Knowledge:** <https://flask.palletsprojects.com/>
- **Groq API Familiarity:** <https://console.groq.com/docs>
- **MongoDB Atlas Documentation:** <https://www.mongodb.com/docs/atlas/>
- **PyPDF2 Library Documentation:** <https://pypdf2.readthedocs.io/>
- **HTML, CSS, and JavaScript Skills:** <https://www.w3schools.com/html/default.asp>
- **Python Programming Proficiency:** <https://docs.python.org/3/>
- **Version Control with Git:** <https://git-scm.com/doc>



- **Development Environment Setup:** <https://flask.palletsprojects.com/en/3.0.x/installation/>



Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for career guidance.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between frontend, backend, database, and AI.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, PyMongo, and Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Resume Analyzer, Skill Gap Evaluator, Dynamic Roadmap Generator, and Certification Advisor.
- **Activity 2.2:** Implement the Flask backend to manage routing, MongoDB operations, and user input processing.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates and asynchronous JavaScript functions to display AI results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Verify the application locally on the development server, testing all core routes and database interactions.

Milestone 6: Conclusion

- **Activity 6.1:** Summarize project achievements, system capabilities, and potential future enhancements.



Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our career guidance needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating roadmaps, skill evaluations, and interview preparation across different roles.

Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select 'API Keys' from the dropdown menu, or go directly to the API Keys page.
- **Step 3 — Create a New API Key:** Click the 'Create API Key' button. Give your key a descriptive name (e.g., career-roadmap-key) so it is easy to identify later.
- **Step 4 — Copy API Key:** Now give a 'Display Name' and 'Expiration' is optional, and Click on 'Submit'. Copy your API key immediately and store it in a safe place.
- **Step 5 — Add Key to Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

```
GROQ_API_KEY=your_copied_api_key_here
```

- **Step 6 — Verify Key Loading:** The config.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
GROQ_API_KEY = os.getenv('GROQ_API_KEY')
```

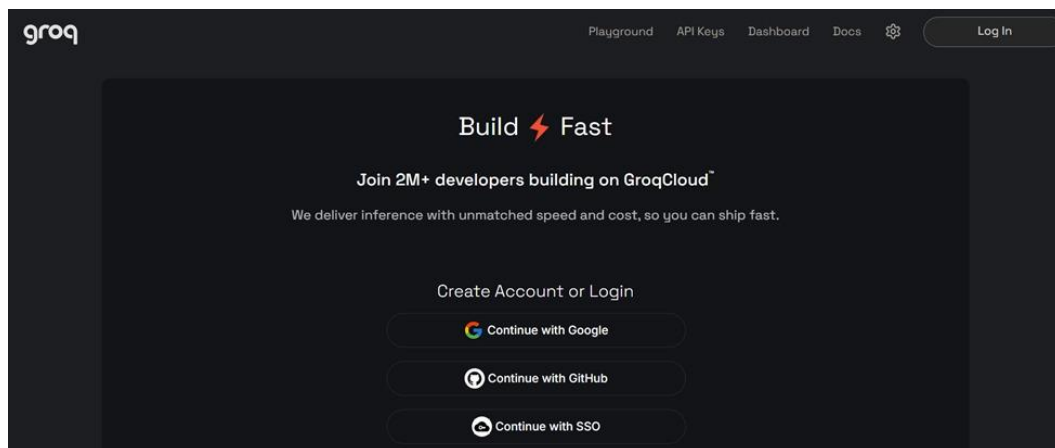


Figure: Groq Console API Key Generation Setup Step 1

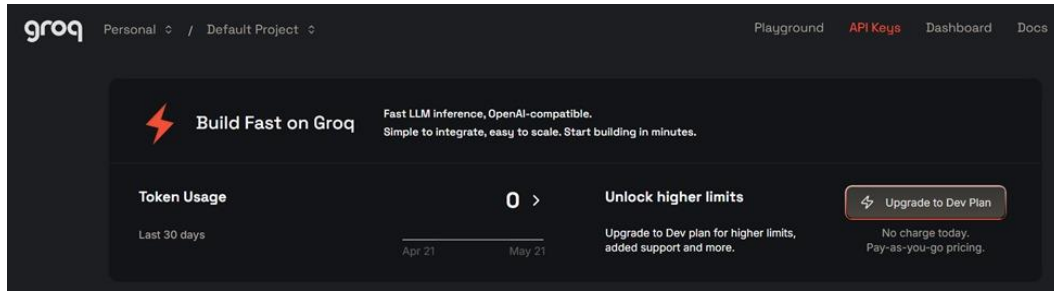


Figure: Groq Console API Key Generation Setup Step 2

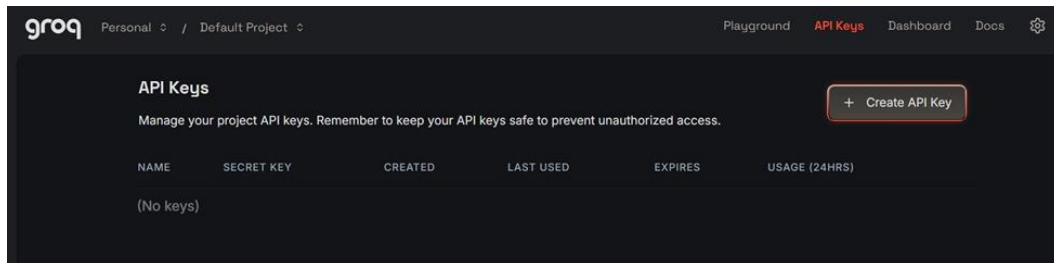


Figure: Groq Console API Key Generation Setup Step 3

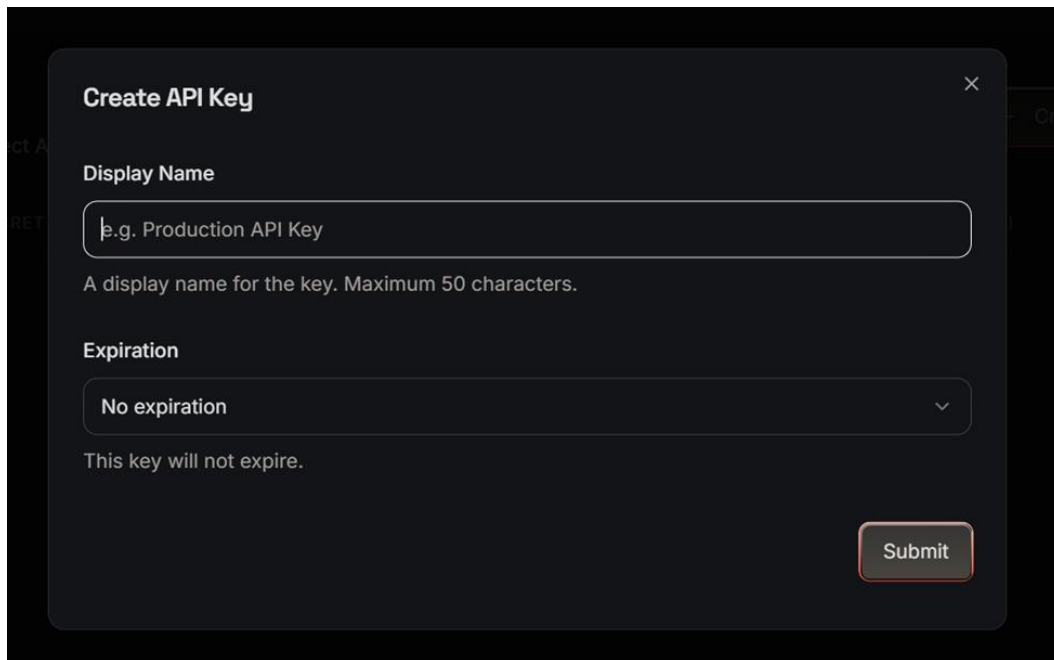


Figure: Groq Console API Key Generation Setup Step 4

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:



- **Understand the Project Requirements:** Review the specific needs of the AI Career Roadmap Generator application, focusing on the types of content it will generate (roadmaps, ATS resume analysis, skill gaps across domains).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to career advisory and analytical tasks.
- **Select the Optimal Model:** Choose llama-3.3-70b-versatile for its balance of creative output quality and generation speed, set with a temperature of 0.5 and max_tokens of 4096 for rich, detailed responses.

Activity 1.3: Define the Architecture of the Application

Draft an Architectural Diagram: Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, MongoDB Atlas database, and Groq API integration.

Detail Frontend Functionality: Outline how users interact with the application entering profile information, selecting target roles, and uploading PDF resumes.

Outline Backend Responsibilities: Specify how the Flask backend handles incoming POST requests, validates user input, builds structured prompts, and communicates with MongoDB and Groq API.

Activity 1.4: Set Up the Development Environment

Install Python and Pip: Ensure Python 3.8+ is installed on your machine along with pip for dependency management.

Create a Virtual Environment & Install Dependencies: Set up a virtual environment and install Flask, PyMongo, Groq SDK, PyPDF2, PyJWT, and python-dotenv.

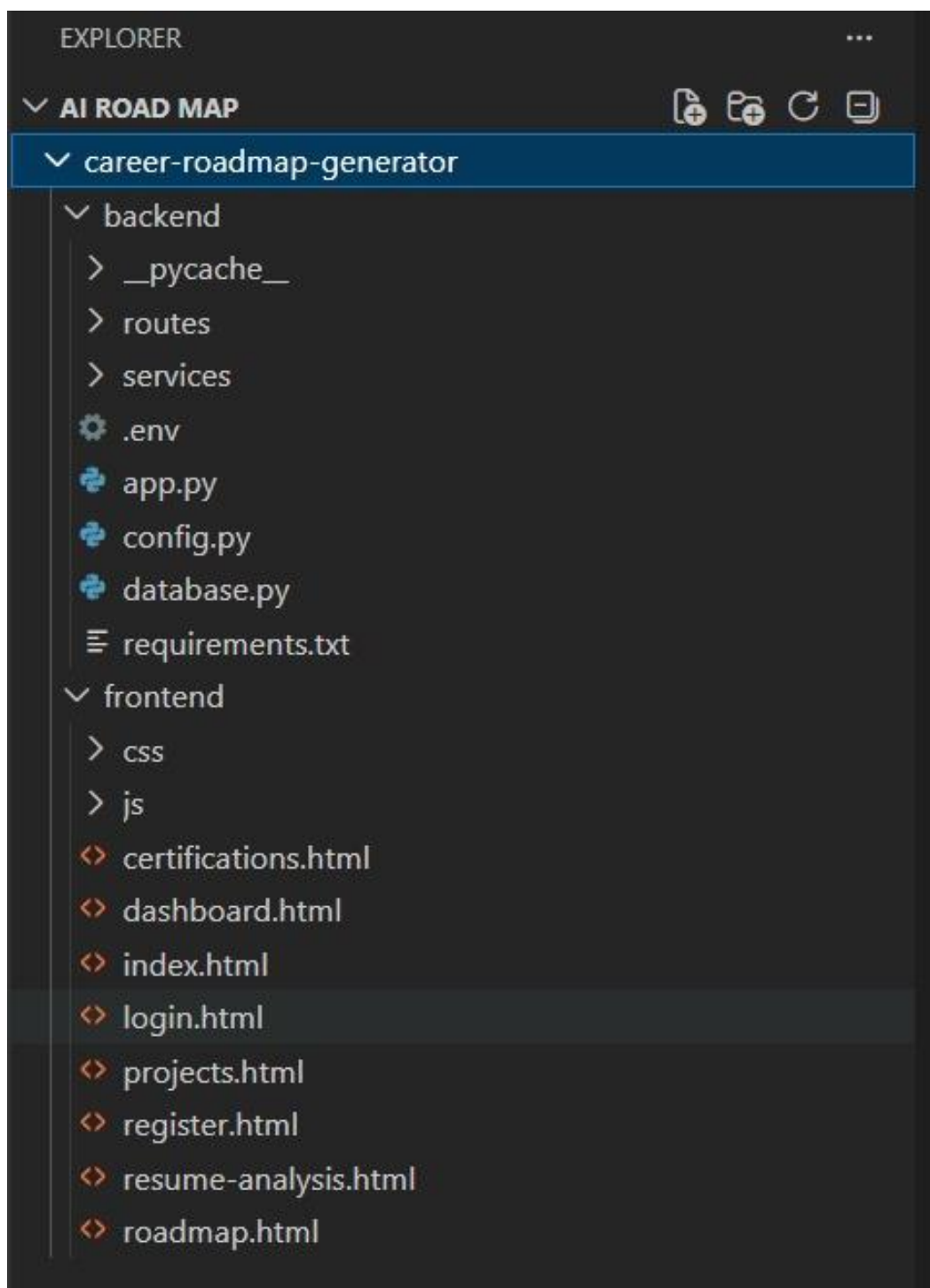


Figure: Project Directory & Environment Structure



Milestone 2: Core Functionalities Development

Milestone 2 focuses on building the underlying service modules that drive career analysis and setting up backend request handling routines.

Activity 2.1: Develop Core Career Advisory Features

Implement the core generation features as structured AI responses driven by specialized system prompts:

- **Resume Analyzer:** Extracts text from PDF files, identifies technical/soft skills, computes ATS scores, and highlights keyword density gaps.
- **Skill Gap Evaluator:** Compares user profile against target roles to identify critical missing skills, tools, and certifications with estimated learning timelines.
- **Dynamic Roadmap Generator:** Generates month-by-month actionable learning paths complete with weekly topics, practice platforms, and portfolio project concepts.

Activity 2.2: Implement the Flask Backend

- **Define Routes in Flask:** Set up Flask Blueprints for auth, resume, roadmap, skills, and certifications endpoints.
- **Process User Input:** Create forms for user inputs, validate data using `request.get_json()`, and enforce maximum file size limits.
- **Integrate API Calls:** Construct structured prompts, invoke `client.chat.completions.create()` with `llama-3.3-70b-versatile`, and parse JSON responses reliably.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of app.py and backend service files, which serve as the backbone of the application. In this milestone, you set up feature functionality through Flask routes, establish prompt construction, and verify API interaction.

Activity 3.1: Writing the Main Application Logic in app.py

Below are the core application initialization and routing implementations as captured during development:

```
"""
Career Roadmap Generator - Main Flask Application.
Production-ready AI-powered career guidance platform.
"""

from flask import Flask, jsonify, send_from_directory
from flask_cors import CORS
from config import Config
import os

# Initialize Flask app
app = Flask(__name__, static_folder='../frontend', static_url_path='')
app.config['MAX_CONTENT_LENGTH'] = Config.MAX_CONTENT_LENGTH

# Enable CORS
CORS(app, resources={r"/api/*": {"origins": "*"}}, supports_credentials=True)

# Register Blueprints
from routes.auth import auth_bp
from routes.resume import resume_bp
from routes.roadmap import roadmap_bp
from routes.skills import skills_bp
from routes.certifications import certifications_bp

app.register_blueprint(auth_bp, url_prefix='/api/auth')
app.register_blueprint(resume_bp, url_prefix='/api/resume')
app.register_blueprint(roadmap_bp, url_prefix='/api/roadmap')
app.register_blueprint(skills_bp, url_prefix='/api/skills')
app.register_blueprint(certifications_bp, url_prefix='/api/certifications')
```

Figure: Backend Application Logic Implementation Part 1

Part 1 illustrates the main Flask application initialization, configuring maximum content length limits (16MB for PDF uploads), setting up CORS policies to allow seamless cross-origin communication, and importing blueprint modules for authentication, resume handling, roadmaps, skills, and certifications.

```
# Serve frontend
@app.route('/')
def serve_index():
    return send_from_directory(app.static_folder, 'index.html')

@app.route('/<path:path>')
def serve_static(path):
    file_path = os.path.join(app.static_folder, path)
    if os.path.isfile(file_path):
        return send_from_directory(app.static_folder, path)
    return send_from_directory(app.static_folder, 'index.html')

# Health check
@app.route('/api/health')
def health_check():
    return jsonify({
        'status': 'healthy',
        'service': 'Career Roadmap Generator API',
        'version': '1.0.0'
    }), 200
```

Figure: Backend Application Logic Implementation Part 2

Part 2 demonstrates the registration of API blueprints under distinct URL prefixes (/api/auth, /api/resume, /api/roadmap, /api/skills, /api/certifications), alongside static file serving logic that renders index.html for root requests.

```
# Error handlers
@app.errorhandler(404)
def not_found(e):
    return jsonify({'error': 'Resource not found'}), 404

@app.errorhandler(413)
def too_large(e):
    return jsonify({'error': 'File too large. Maximum size is 16MB'}), 413

@app.errorhandler(500)
def server_error(e):
    return jsonify({'error': 'Internal server error'}), 500

if __name__ == '__main__':
    print(f"""
    🚀 AI Career Roadmap Generator API Server

    Server running on http://localhost:{Config.PORT}
    Frontend: http://localhost:{Config.PORT}
    API Base: http://localhost:{Config.PORT}/api
    Health:   http://localhost:{Config.PORT}/api/health

    """)
    app.run(
        host='0.0.0.0',
        port=Config.PORT,
        debug=Config.DEBUG
    )
```

Figure: Backend Application Logic Implementation Part 3

Part 3 depicts the API health check endpoint (`/api/health`) returning service status metrics, error handler definitions for 404, 413, and 500 status codes, and the server startup execution block running on port 5000.

Core Route Handlers & Helpers Breakdown

For each API route, logic is implemented to read JSON input via `request.get_json()`, validate user credentials using JWT authentication middleware (`@token_required`), construct tailored AI prompts in `groq_service.py`, execute inference via Groq's LLaMA 3.3-70B model, parse JSON output safely, and update MongoDB Atlas collections.



Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for AI Career Roadmap Generator. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated roadmaps, skill gaps, and resume analyses based on user interactions.

Activity 4.1: Designing and Developing the User Interface

- **Set Up Base HTML Structure:** Develop HTML interfaces including headers with brand logos, navigation bars, input forms, and dynamic results containers.
- **Design Responsive Layout Using CSS:** Create CSS stylesheets implementing a modern dark glassmorphism aesthetic with flexbox and grid layouts, ensuring responsiveness across desktop and mobile screens.
- **Create UI Components:** Build modular UI cards for roadmap milestones, skill progress bars, certification lists, and interactive chat panels.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

- **Handle Form Submission Asynchronously:** Attach submit event listeners to form inputs, sending data via asynchronous Fetch API requests to backend endpoints while displaying animated loading spinners.
- **Render Results Dynamically:** Dynamically build and inject DOM elements to render month-by-month roadmaps, ATS scorecards, and project recommendations upon receiving API responses.
- **Restore UI State:** Re-enable action buttons and restore original UI state after execution completes.

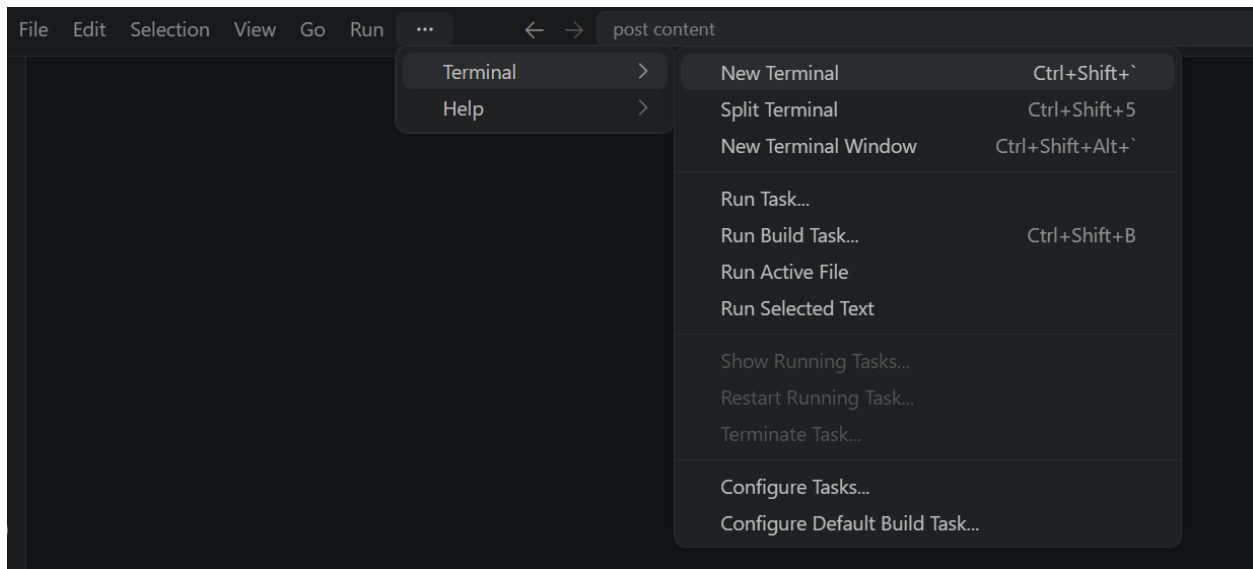
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the application locally to verify correct operation, configuring server environments, managing dependencies, and validating database persistence.

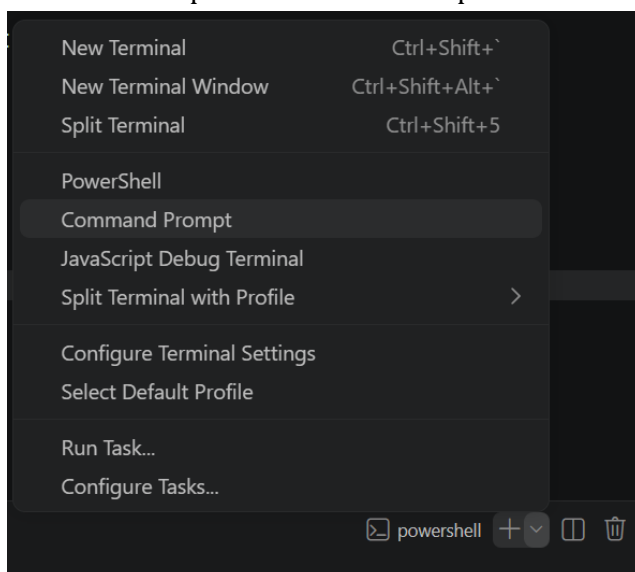
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:
GROQ_API_KEY=your_groq_api_key_here

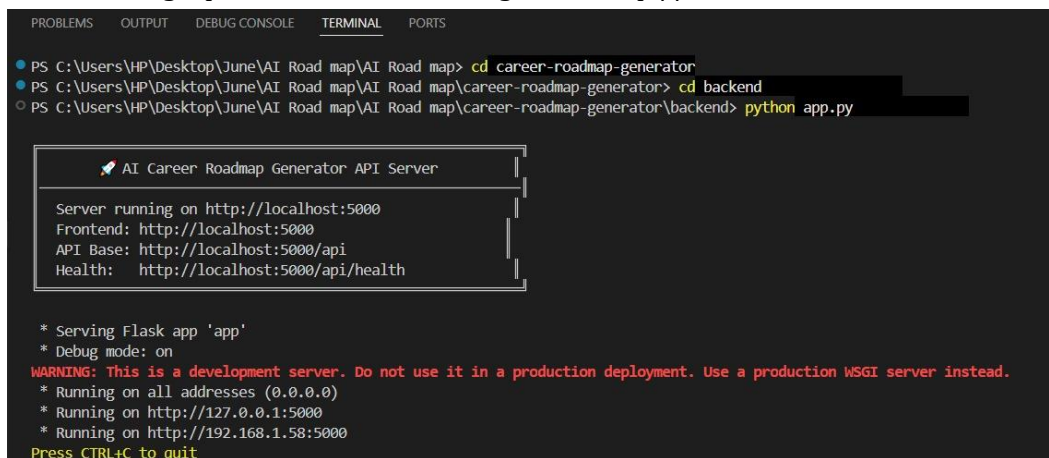
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\HP\Desktop\June\AI Road map\AI Road map> cd career-roadmap-generator  
PS C:\Users\HP\Desktop\June\AI Road map\AI Road map\career-roadmap-generator> cd backend  
PS C:\Users\HP\Desktop\June\AI Road map\AI Road map\career-roadmap-generator\backend> python app.py  
  
AI Career Roadmap Generator API Server  
Server running on http://localhost:5000  
Frontend: http://localhost:5000  
API Base: http://localhost:5000/api  
Health: http://localhost:5000/api/health  
  
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://192.168.1.58:5000  
Press CTRL+C to quit
```

- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.



Exploring the Website's Web Pages:

Home / Landing Page

Description: The main interface serves as the entry point, featuring a dark glassmorphism header, brand logo, tagline, and quick-navigation links to all career guidance modules.

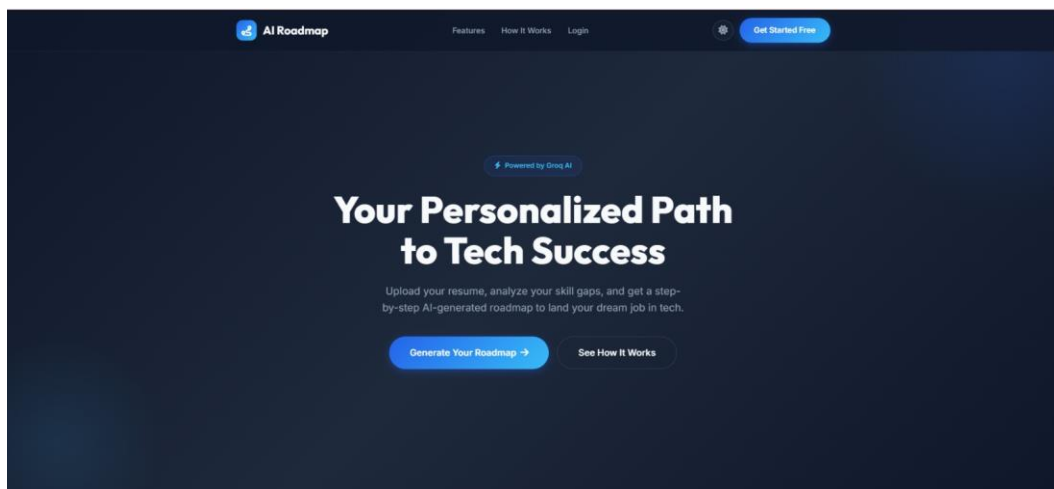


Figure: Home / Landing Page

Sign In Page

Description: A secure authentication interface allowing registered users to enter their credentials and receive a JWT session token for protected route access.

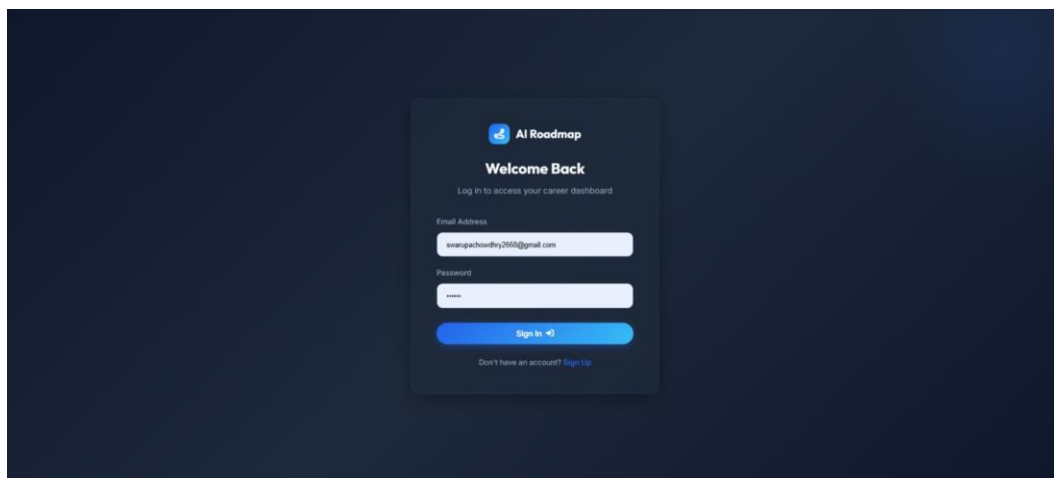




Figure: Sign In Page

Sign Up Page

Description: Registration portal where new users create accounts, specify their primary career goals, and initialize their profile in MongoDB Atlas.

The image shows a 'Create Account' form for 'AI Roadmap'. The form is centered on a dark blue background. It includes fields for 'Full Name' (VEMULAPALLI BINDU SWARUPA), 'Email Address' (swarupachowdhry2668@gmail.com), 'Target Role (Optional)' (Cloud Architect), and 'Password' (masked with dots). A 'Create Account' button is at the bottom, with a link to 'Sign In' for existing users.

Figure: Sign Up Page

Career Overview & Analytics Dashboard

Description: Displays overall career readiness metrics, skill strength scores, and quick progress summaries generated by the AI analytics engine.

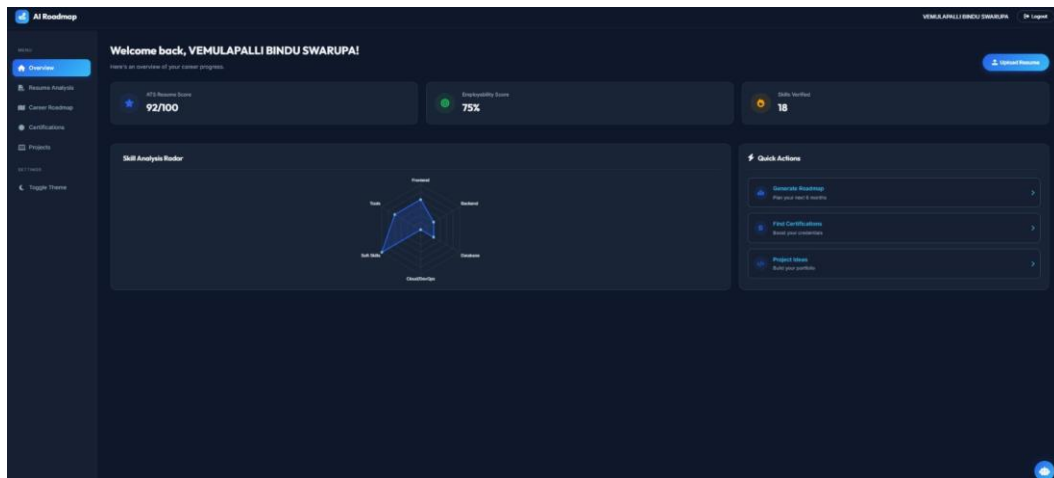


Figure: Career Overview & Analytics Dashboard

Career Roadmap View - Part 1

Description: Presents a visual month-by-month breakdown of foundational skills, focus areas, and milestones tailored to the user's target job role.

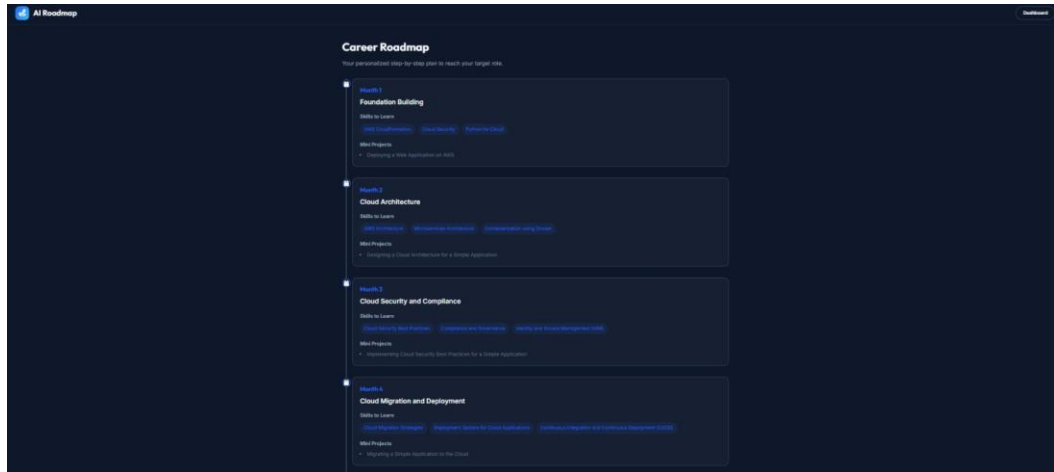


Figure: Career Roadmap View - Part 1

Career Roadmap View - Part 2

Description: Detailed weekly schedule view showing specific practice platforms, recommended mini-projects, and milestone completion checkboxes.

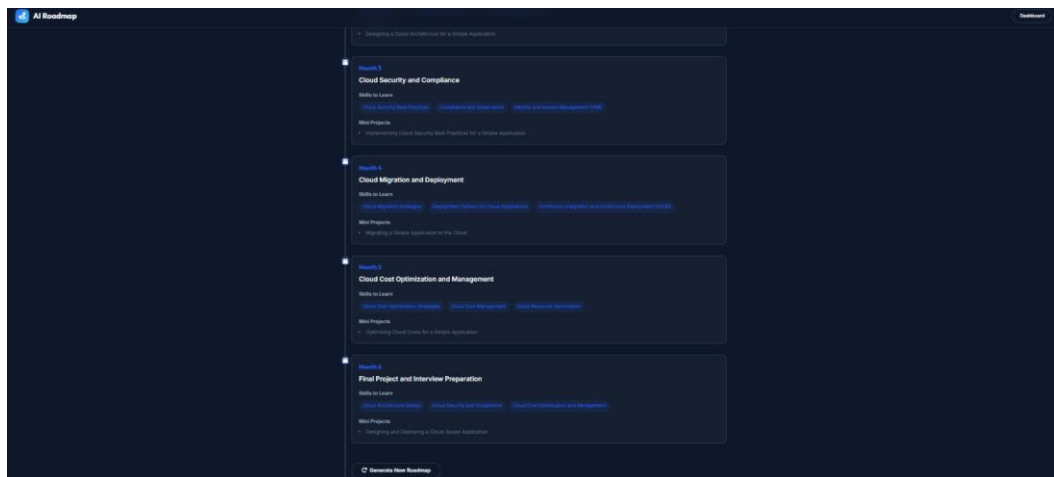


Figure: Career Roadmap View - Part 2

Portfolio Projects Page

Description: Recommends portfolio project builds categorized into Beginner, Intermediate, and Advanced difficulty tiers with required tech stacks and feature outlines.

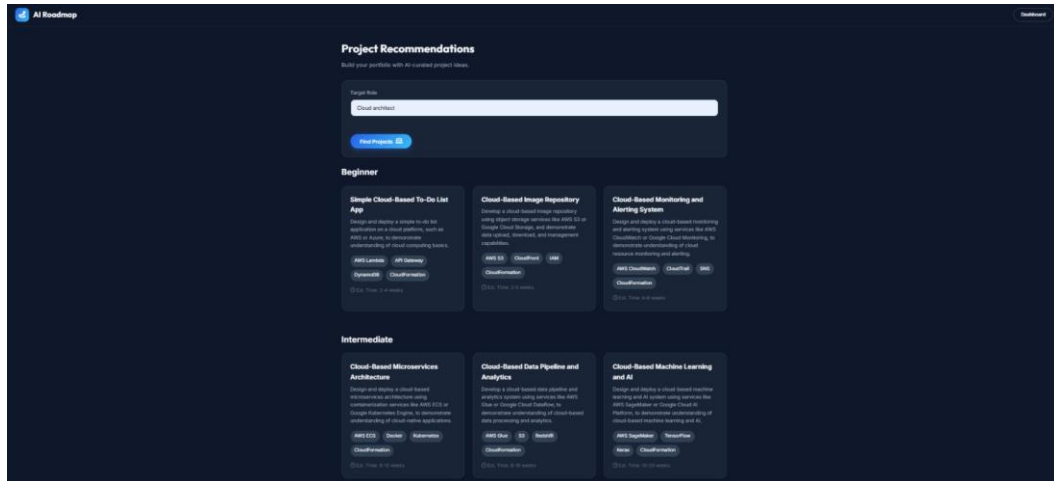


Figure: Portfolio Projects Page

Certifications Advisory Page

Description: Displays recommended industry certifications with cost estimates, provider details, prerequisites, and estimated ROI scores.

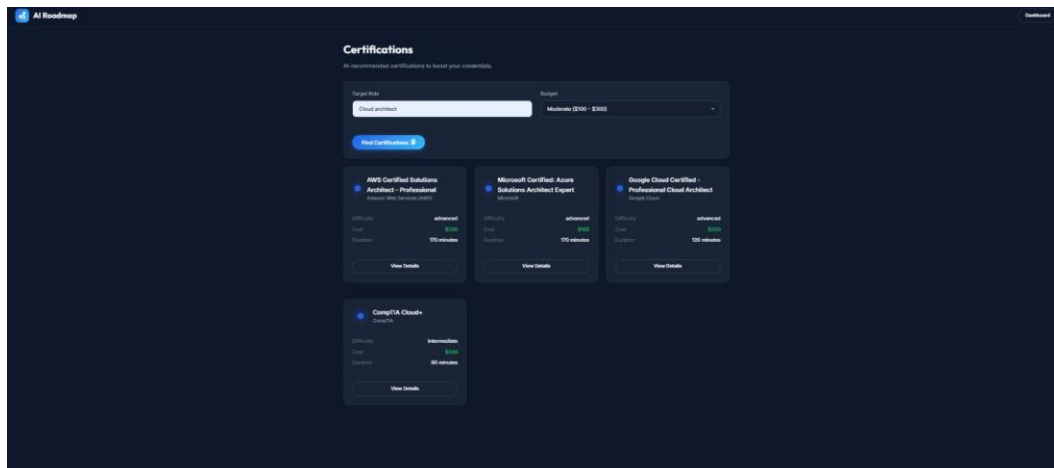


Figure: Certifications Advisory Page

Resume ATS Analyzer Page

Description: Allows users to upload PDF resumes, displaying calculated ATS compliance scores, extracted skills, and keyword density feedback.

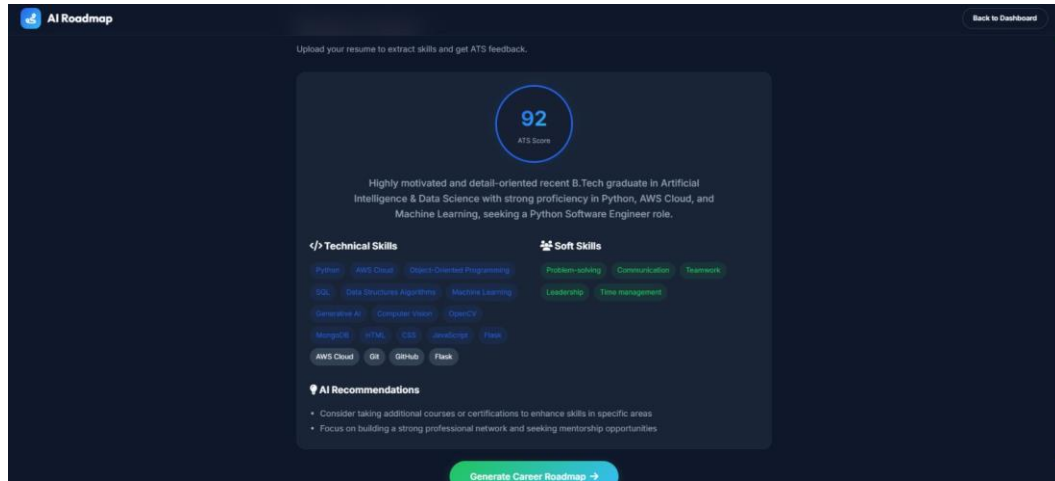


Figure: Resume ATS Analyzer Page



Conclusion:

AI Career Roadmap Generator is a comprehensive generative AI platform built with Flask, MongoDB Atlas, and styled with a glassmorphism UI that streamlines career planning and skill development. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate tailored roadmaps, ATS resume analyses, skill gap evaluations, and interview preparation content. The project highlights how targeted AI and a structured framework can boost career growth for job seekers and professionals, with significant potential for future scalability into live job market integrations and voice-based mock interviews.